

MPC stage briefing: GNN-based model predictive control of multi-hit hot forging (state as of 2026-10-08)

Purpose: a self-contained technical summary of the MPC (model predictive control) stage of the Agility Forge project, for an agent that has not seen the codebase. Repository root: `/home/exr343/CIRP_2027`. All paths below are relative to `applications/Agility_Forge/` unless they start with `GNN/`, `control/`, etc. under that folder, or with `jax_forge/`. Code is run as modules from the repository root (e.g. `python -m applications.Agility_Forge.control.mpc_coil ...`).

1. The problem

- **Process:** a steel billet (round bar, 15.875 mm diameter, 101.52 mm long, x from -5 mm at the clamp to 96.52 mm at the free end) is forged hot into a square rod by many die hits.
- **Simulator** (the "plant", ground truth): a differentiable thermo-mechanical finite-element model (JAX, `jax_forge/`, coupled temperature-dependent J2 plasticity). About 7–9 minutes of wall time per hit, plus a few minutes per reheat.
- **Die:** a flat die pair, 12.7 mm wide along the bar, fixed in space. A hit presses the bar between two parallel plates.
- **Hit controls** (one hit = 3 numbers):
 - **die centre** c (mm): where along the bar, in current, deformed coordinates;
 - **angle** θ (degrees): rotation of the press direction about the bar axis, 0° presses along y , 90° along z ;
 - **stroke** u (mm): how far each plate moves inward from first contact.
 - Each hit lasts 0.8 s.
- **Reheat:** every 6 hits an induction coil, 24.5 mm long, reheats the bar.
 - The coil profile is 1096 °C over its length, falling by 5.4545 °C per mm outside it.
 - Each surface and volume point keeps the **hotter** of its current temperature and the coil profile.
 - On the very first cycle (fresh billet), the temperature is set to the coil profile.
- **Target** (`control/targets/ideal_square_10.6/`), cold nominal dimensions:
 - round (unchanged) from the clamp to $x = 17.73$ mm, the 800 °C point of the starting temperature profile;
 - a 7.5 mm taper;
 - a 10.6×10.6 mm square from 25.23 to 153.15 mm, its length set by volume conservation.
 - `target_on_billet.vtu` gives every one of the 4355 surface nodes its own target displacement. The mapping is lengthwise by volume conservation and radial across the section.

- **Forging limit:** 800 °C. Colder metal is considered too cold to forge.

2. The surrogate (GNN) the MPC plans with

- **Code:** `GNN/coil_T.py`. MeshGraphNets-style encode–process–decode network in PyTorch.
- **Checkpoint used by the MPC:** `GNN/coil_T_sweep_test_square/mp_5/stage3.pt`, with **M = 5** message-passing steps (847,108 parameters). The plain square run was held out of training.
- **Graph:** the billet's surface mesh, 4355 nodes and 26,118 directed edges (both directions of every surface-triangle edge). The topology is fixed; only positions change.
- **Node state:** displacement (3) and **temperature** (1). Temperature has been part of the state since 2026-10-07.

Node inputs, 16 per node:

Columns	Inputs	Scaling
0–3	is clamp face, is pin node, is lateral wall, reheat flag	raw
4–6	die push: the displacement the die would impose on the node	normalized
7	distance along the bar to the die centre	normalized
8–10	displacement	normalized
11	temperature	normalized
12	temperature jump at a reheat ÷ 300	raw
13–14	sin and cos of the hit angle	raw
15	stroke fraction, $(u - 0.5)/1.5$	raw

The die push is computed the way the plant places its dies: the dies start at the outermost lateral-wall nodes under the 12.7 mm footprint and close in by the stroke. The footprint has a soft 0.25 mm edge so the gradient with respect to die position exists.

Edge inputs, 8 per edge: the edge vector and its length, in the undeformed billet and in the current shape.

Outputs, 4 per node: the change in displacement (x, y, z) and the change in temperature over one hit.

- For a reheat step, the GNN predicts only the shape change.
- The temperature after a reheat is set by the exact rule. It matches the simulator to 0.000 °C on the reheats checked.

Accuracy on the held-out plain square run: whole cycles predicted from the true state at each scan.

Measure	Value
Hausdorff distance to the simulated surface	0.714 mm
Chamfer distance	0.124 mm ²
Mean temperature error, whole surface	9.7 °C

Mean temperature error, under the die	39.8 °C
Time for one gradient of a 6-hit forecast	0.072 s

Definitions:

- **Hausdorff:** the largest distance from any point of one surface to the other.
- **Chamfer:** the mean squared nearest-point distance, summed in both directions.

Training: three stages, all in `GNN/coil_T.py`:

1. old random hits from an earlier simulator version;
2. old scheduled square runs;
3. the new simulator's 5-pass square runs, `data/dataset_diel2_coil_5pass/`. Training here uses whole cycles (a reheat plus 6 hits) fed with the model's own predictions.

3. The MPC loop (`control/mpc_coil.py`)

One cycle:

1. **Scan:** read the bar's surface displacement and its **real surface temperature** from the simulator. In practice this would be a 3D scanner and an IR camera.
2. **Stop check:** stop if every 2 mm slice from $x = 31.58$ mm to 4 mm before the free end has both its 0° and 90° widths ≤ 10.8 mm ($10.6 + 0.2$), or after 20 cycles. The slices next to the taper are excluded on purpose: no data run ever got them below 11–13 mm.
3. **Plan:** choose 18 numbers, (c, θ, u) for each of the 6 hits (section 4).
4. **Reheat:** coil centre = the average of the 6 planned die centres. The plant applies the keep-the-hotter rule through a temperature ramp, retried at 50, then 25, then 12.5 °C per ramp step if the solver fails.
5. **Hit:** apply **all 6** planned hits to the simulator. A failed hit is retried once as two half-strokes.
6. Repeat.

The full 6-hit plan is always executed; there is no receding horizon inside a cycle. The run checkpoints after every hit and reheat, and resumes when rerun with the same output folder.

Forecast inside the planner (`forecast`, and `forecast_cost_batch` for batches):

1. Temperature after the reheat:
 - $T^+ = \text{smoothmax}(T_{\text{scan}}, \text{coil}(\bar{c}))$;
 - $\text{coil}(\bar{c}) = 1096 - 5.4545 \cdot \text{softplus}(|x - \bar{c}| - 12.25)$, with $\bar{c}$ the average die centre;
 - on the first cycle, $T^+ = \text{coil}(\bar{c})$;
 - the smooth versions (a soft maximum with 5 °C softness, and a softplus edge about 0.5 mm wide) are used while planning; the exact rule is used for the stored forecast.
2. Shape after the reheat: the GNN's reheat step (skipped on the first cycle).

3. Hits 1–6: $(x, T) \leftarrow \text{GNN}((x, T), \text{hit } k)$. The predicted temperature is carried forward from hit to hit.

Cost (unchanged since 2026-09-27 experiments, "e14"):

$$J = \sum_{k=1..6} \sum_{\text{surface nodes } i} [(y_i^k - y_i^{\text{target}})^2 + (z_i^k - z_i^{\text{target}})^2]$$

- These are the GNN-predicted node positions after each of the 6 hits.
- Only the cross-section (y, z) is penalized; length is left to volume conservation.
- Temperature is **not** in the cost.

Bounds:

- die centre from 31.58 mm (the die edge at the start of the square section) to (scanned free end – 9.35 mm);
- angle 0–180°;
- stroke 0.5–2 mm.

There are no other constraints. A ± 11 mm "coil zone" constraint, keeping the die centres near the coil centre, was used in the first run and removed on 2026-10-08 by the user's decision.

4. The planner (since 2026-10-08): random shooting + top-K gradient refinement

Modelled on RoboCraft (Shi et al., RSS 2022, arXiv:2205.02909, Sec. III-E.2). Class `Planner` in `control/mpc_coil.py`:

Stage	What it does	Settings
A, screening	Draw N plans uniformly in the bounds, seeded per cycle with <code>[seed, cycle]</code> ; score each with the batched GNN forecast and the cost J ; gradients off	$N = 100$, batch $B = 50$, seed 0
B, selection	Keep the K lowest-cost plans, sorted	$K = 5$
C, refinement	From each of the K starts, an independent SLSQP solve (scipy). Variables rescaled to $[0, 1]$ inside the bounds, cost divided by its value at the start, gradient by autograd through the 7 GNN steps	maxiter 100, scipy default tolerance
D, output	Return the refined plan with the lowest GNN cost; save diagnostics in the manifest	–

• **Diagnostics saved per cycle:**

- all N screening costs and the K chosen indices;
- per start: initial and final cost, iterations, function evaluations, termination message, starting-gradient norm, and how many variables end on a bound;
- the time of each stage.

• **Command-line options:**

- `--n-samples 1` gives the old single-start planner, whose start is the training data's pattern: the next 3 stations, 10.73 mm apart, each hit at 0° then 90°, stroke 1.5 mm.

- `--sample-train-range` (off) samples only inside the training data's action ranges: die centre 30.6–159.5 mm, angle –5.0 to 95.0°, stroke 0.12–2.0 mm.
- `--include-warm-start` (off) lets the previous cycle's plan replace the worst start.

- **Deterministic GPU mode** (`torch.use_deterministic_algorithms(True)`, `CUBLAS_WORKSPACE_CONFIG=:4096:8`) is on. Without it, run-to-run floating-point differences make SLSQP return different plans for the same seed. It costs about 20% in speed.
- **Checks** (`control/ablation_multistart.py`), all passing:
 - batched forecast = one-at-a-time forecast (inputs identical; cost within $1.8\text{e-}7$ relative);
 - $N = K = 1$ reproduces the previous planner exactly;
 - the same seed gives the same plan;
 - gradients are finite and non-zero.

GNN-only comparison on 6 held-out planning problems: plain square run scans before hits 1, 13, 23, 35, 47 and 62.

Method	Mean final cost relative to the single default start	Wins vs default (of 6)	Planning time per cycle
Top-5 of $N = 200$	0.962	5	48.1 s
5 uniform random starts, no screening	0.975	5	37.2 s
Single default start	1.000	–	7.6 s

- The default start is better than both multi-start methods on the fresh billet.
- The screening rank barely predicts the refined result: the best refined plan came from the best-screened sample in only 1 of 6 problems.
- SLSQP needs about 80–90 function evaluations per solve.
- One GNN rollout (reheat plus 6 hits) takes 0.039 s alone, or 0.029 s per plan in a batch of 50, on an L40S GPU.
- Full report: progress/control_reports/2026-10-08_multistart_planner/.

5. Run history and results

Run 1 (`control/results/coil_mpc_tstate_M5/`, job 3984018):

- **Setup:** single default start, ± 11 mm coil-zone constraint, any angle. **Stopped by the user after 64 hits** (11 cycles).

Measured from the run's files at hit 62, against the scheduled (non-MPC) plain square run at the same hit:

Measure	MPC run 1	Scheduled run
---------	-----------	---------------

Hausdorff distance to target	23.4 mm	13.5 mm
Chamfer distance to target	43.5 mm ²	11.1 mm ²
Bar centreline offset at x = 110 mm (bending; size of the y, z offset of the cross-section centre)	3.2 mm	0.26 mm

Cause: the optimizer exploited GNN error.

- From cycle 5 on, plans stacked 4–6 hits on one station (31.6 mm). The training data never had that: it always has 2 hits per station per cycle, 0° then 90°.
- Cycles 7–8 used off-axis angles (51°, 123°, 135°, 179°).
- Over cycles 1–10 the GNN predicted the cost would fall by 39,618; it actually rose by 11,392.
- The repeated clamp-end hits bent the bar, and chilled the hammered zone below 800 °C (about 650–750 °C).
- The GNN's per-hit forecast error rose from about 0.5–1 mm to about 4 mm.

Run 2 (control/results/coil_mpc_tstate_M5_multistart/, job 3990381):

- **Setup:** the multi-start planner (N = 100, K = 5), no coil zone, any angle, 20-cycle cap, M = 5.
- **Status on 2026-10-08 15:16:** queued, not started; estimated start about 18:40.

After every MPC run: `slurm_scripts/submit_animation_mpc.sh` renders two GIFs into the run folder: the simulation alone, and the simulation with the GNN forecast underneath. Run 2's GIFs are job 3990382 (afterany).

6. Data and file formats

- **Run folder:** `manifest.json`, `rollout_01/`, and `failed_hits.jsonl` if any attempt failed.
- **manifest.json**, in the data generator's format:
 - `records`: one per hit (`kind: hit_final`), with `die centre`, `angle`, `stroke`, `cycle`, `vtu_path`, `forecast_rmse_mm` (GNN vs plant node RMSE), `forecast_T_abs_err_C`, `free_end_mm` and `wall_time_s`;
 - `reheats`: one per reheat, with `coil_center_mm`, `vtu_path`, and the maximum temperature jump and number of ramp steps;
 - `cycles`: one per plan, with the plan, initial and final cost, iterations, `plan_time_s`, `worst_width_at_scan_mm`, and `multistart` diagnostics;
 - `stop`: why and when the run stopped.
- **rollout_01/:**
 - one `.vtu` per hit and reheat, with full-mesh `Displacement` and `Temperature` fields;
 - `cycle_XX_forecast.npy` ($6 \times 4355 \times 3$, the GNN's predicted surface displacement after each hit) and `cycle_XX_forecast_T.npy` (6×4355);
 - `checkpoint.npz`, the full solver state, used to resume.

- **Surface nodes:** `build_surface_mesh_info(...).full_to_surface` maps full-mesh nodes to the 4355 surface nodes, in the order the GNN and the target use.

7. Key files

File	Role
<code>control/mpc_coil.py</code>	MPC loop, forecast, cost, multi-start <code>planner</code> , batched forecast, stop rule
<code>control/plant_interface.py</code>	<code>ForgingPlant</code> : <code>reset</code> , <code>reheat(state, T_new, hit)</code> , <code>step(state, hit, reheat=False)</code> ; <code>coil_temperature</code> , <code>reheat_temperature</code> ; <code>linear_solver="scipy"</code> (direct) is required for late reheats
<code>control/ablation_multistart.py</code>	planner checks and the GNN-only comparison
<code>GNN/coil_T.py</code>	the temperature-state GNN: features (<code>node_features</code>), model (<code>CoilGNN.step</code> , <code>step_shape</code>), training
<code>control/targets/ideal_square_10.6/</code>	target files and <code>target_spec.json</code>
<code>generate_square_coil_rollout.py</code>	the scheduled data runs (the non-MPC baseline): 5 passes over 10.73 mm stations, 0° then 90° per station
<code>output/diel2_coil_animation/</code> <code>animate.py</code> , <code>animate_mpc.py</code>	the run GIFs
<code>slurm_scripts/submit_mpc_coil.sh</code>	<code>CKPT=<.pt> OUT=<dir> sbatch --export=ALL ...</code>
<code>progress/</code>	all reports (index in <code>progress/README.md</code>)

8. Known issues and open questions

1. **Model exploitation.** The optimizer finds hit patterns where the GNN is wrong: repeated hits on one spot, and off-axis angles. Mitigations that have not been tried:
 - restrict the plans to the data's pattern, 2 hits per station at 0° and 90° $\pm$ 10° with stations about 10 mm apart (the user declined the angle limit for runs 1 and 2);
 - sample and optimize only inside the training data's ranges;
 - retrain the GNN on the MPC's own hits (the runs are saved in the training-data format for that).
2. **Myopic planning.** The cost only looks one cycle (6 hits) ahead. Nothing in it rewards working on parts of the bar that won't improve within this cycle, or penalizes bending that hurts later.
3. **Cold metal.** With the coil zone removed, hits can land far from the coil. The 800 °C limit is not enforced. The GNN predicts temperature, so a constraint like "predicted surface temperature under the die $\geq$ 800 °C" is possible.
4. **The scheduled baseline doesn't reach the target either.** The plain square run (pass-5 half-gap 5.0 mm, the hardest squeeze of the 10 runs) ended after 103 hits with the 0° width at about 11.1–11.4 mm and the 90° width at about 10.4–10.6 mm. Each 90° hit pushes the 0° sides back out.

5. **Compute:** about 15–20 hours per 20-cycle MPC run, almost all of it simulation. GPU queue waits on the cluster can be hours.

9. Conventions the user expects

- Always say "message-passing steps (M)", never "depth".
- After every MPC run, make and show the GIFs.
- Answers: short, plain sentences; no ASCII diagrams; a PDF for longer reports.
- Don't overwrite or delete data without asking.